LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 3
Student Name: B. Pranathi
Roll No.: A24126552071
Date: 30-08-2026

1. TITLE

Student Profile Webpage using HTML, CSS, and JavaScript DOM Manipulation

2. AIM

To create an interactive Student Profile webpage using HTML, CSS, and JavaScript that accepts student details and dynamically displays the profile using a JavaScript class, DOM manipulation, and event handling.

3. OBJECTIVES

• To design a simple and responsive student profile form using HTML and CSS.

• To create a Student class in JavaScript using a constructor.

• To accept student name, roll number, department, and CGPA from input fields.

• To access HTML elements using document.getElementById().

• To handle button clicks using addEventListener().

• To validate that all input fields are filled.

• To dynamically create and display the student profile using DOM methods.

4. THEORY

HTML is used to create the structure of the Student Profile webpage, while CSS is used for layout, spacing, colors, borders, shadows, and button styling. JavaScript adds dynamic behavior to the webpage. A JavaScript class is used to represent a student object containing name, roll number, department, and CGPA. The DOM (Document Object Model) allows JavaScript to access input elements and create or modify HTML elements dynamically. Event handling is used to execute the profile display operation when the Show Profile button is clicked.

5. PROCEDURE / ALGORITHM

1. Create an HTML document with head and body sections.

2. Define internal CSS for the page container, heading, labels, input fields, button, and profile display.

3. Create input fields for Name, Roll Number, Department, and CGPA.

4. Create a Show Profile button and a profile display area.

5. Define a Student class with a constructor for student details.

6. Select the button and profile display area using document.getElementById().

7. Add a click event listener to the Show Profile button.

8. Read and trim the values entered by the user.

9. Validate that all fields contain values.

10. Create a Student object using the entered details.

11. Create profile elements dynamically using createElement() and display them in the webpage.

12. Save the HTML file and open it in a browser to verify the output.

7. Add a click event listener to the Show Profile button.

8. Read and trim the values entered by the user.

9. Validate that all fields contain values.

10. Create a Student object using the entered details.

11. Create profile elements dynamically using createElement() and display them in the webpage.

12. Save the HTML file and open it in a browser to verify the output.

6. PROGRAM

```
<!DOCTYPE html>
<html>
<head>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f5f5f5;
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 100vh;
            padding: 20px;
        }

        .container {
            max-width: 450px;
            width: 100%;
            background: white;
            padding: 30px;
            border-radius: 10px;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1);
        }

        h2 {
            text-align: center;
            color: #1a2e3f;
            margin-bottom: 20px;
        }

        label {
            display: block;
            font-weight: 600;
            font-size: 14px;
            margin-top: 12px;
            margin-bottom: 4px;
        }

        input {
            width: 100%;
            padding: 10px;
            border: 1px solid #ddd;
            border-radius: 5px;
            font-size: 14px;
            box-sizing: border-box;
        }

        button {
            width: 100%;
            padding: 12px;
            margin-top: 20px;
            background: #007bff;
            color: white;
            border: none;
            border-radius: 5px;
            font-size: 16px;
            cursor: pointer;
        }

        button:hover {
            background: #0056b3;
        }

        .profile {
            margin-top: 20px;
            padding: 15px;
            background: #f9f9f9;
            border-radius: 5px;
            border-left: 4px solid #007bff;
        }

        .profile p {
            margin: 8px 0;
        }

        .label {
            font-weight: bold;
            display: inline-block;
            width: 100px;
```

```
        }

        .empty {
            text-align: center;
            color: #999;
            padding: 20px;
        }
    </style>
</head>
<body>

<div class="container">
    <h2>Student Profile</h2>

    <label>Name</label>
    <input type="text" id="name" placeholder="Enter name">

    <label>Roll Number</label>
    <input type="text" id="roll" placeholder="Enter roll number">

    <label>Department</label>
    <input type="text" id="dept" placeholder="Enter department">

    <label>CGPA</label>
    <input type="text" id="cgpa" placeholder="Enter CGPA">

    <button id="showBtn">Show Profile</button>

    <div id="profileDisplay">
        <div class="empty">No profile yet</div>
    </div>
</div>

<script>
    // Student class
    class Student {
        constructor(name, rollNo, department, cgpa) {
            this.name = name;
            this.rollNo = rollNo;
            this.department = department;
            this.cgpa = cgpa;
        }
    }

    // DOM selection
    const showBtn = document.getElementById('showBtn');
    const profileDisplay = document.getElementById('profileDisplay');

    // Event handling
    showBtn.addEventListener('click', function() {
        // Get user input
        const name = document.getElementById('name').value.trim();
        const rollNo = document.getElementById('roll').value.trim();
        const department = document.getElementById('dept').value.trim();
        const cgpa = document.getElementById('cgpa').value.trim();

        // Validation
        if (!name || !rollNo || !department || !cgpa) {
            alert('Please fill in all fields.');
            return;
        }

        // Create Student object
        const student = new Student(name, rollNo, department, cgpa);

        // Clear and display profile
        profileDisplay.innerHTML = '';

        const profileDiv = document.createElement('div');
        profileDiv.className = 'profile';

        const details = [
            ['Name', student.name],
            ['Roll No', student.rollNo],
            ['Department', student.department],
            ['CGPA', student.cgpa]
        ];

        details.forEach(function([label, value]) {
            const p = document.createElement('p');
```

```
            p.innerHTML = `<span class="label">${label}</span> : ${value}`;
            profileDiv.appendChild(p);
        });

        profileDisplay.appendChild(profileDiv);
    });
</script>

</body>
</html>
```

7. CODE EXPLANATION

Element / Property	Purpose
class Student	Defines a Student class to store student details.
constructor()	Initializes name, roll number, department, and CGPA.
document.getElementById()	Selects HTML elements using their ID.
addEventListener('click', ...)	Executes the profile display code when the button is clicked.
value.trim()	Reads the input value and removes extra spaces.
Validation	Checks whether all four fields contain values.
new Student(...)	Creates a Student object using the entered details.
innerHTML	Clears the previous profile and inserts the required HTML.
createElement()	Creates new HTML elements dynamically.
appendChild()	Adds dynamically created elements to the profile area.
.profile / .label	CSS classes used to style the displayed student information.
:hover	Changes the button appearance when the mouse pointer is over it.

8. TEST CASES

Test Case	Action	Expected Result	Status
TC-01	Open HTML file	Student Profile webpage loads successfully	Pass
TC-02	Click Show Profile with empty fields	Alert asks the user to fill in all fields	Pass
TC-03	Enter student details and click Show Profile	Student profile is displayed with Name, Roll No, Department, and CGPA	Pass
TC-04	Click Show Profile again with new details	Previous profile is cleared and the new profile is displayed	Pass
TC-05	Move mouse over Show Profile button	Button hover styling is applied	Pass

9. OUTPUT

The browser displays a centered Student Profile form with four input fields, a blue Show Profile button, and a profile card below the button. After entering the following sample values and clicking the button, the displayed profile is:

Field	Output
Name	Pranathi
Roll No	A24126552074
Department	CSM
CGPA	8.5

Output Display: Student Profile → Name: Pranathi | Roll No: A24126552074 | Department: CSM | CGPA: 8.5

10. RESULT

Thus, the Student Profile webpage was successfully created using HTML, CSS, and JavaScript. The program accepts student details, creates a Student object, validates the input, and dynamically displays the student profile in the browser. The output was verified successfully.